

RESEARCH

Open Access



A ground truth approach for assessing process mining techniques

Dominique Sommers^{1*}, Natalia Sidorova¹ and Boudewijn van Dongen¹

Competing interests for Boudewijn van Dongen are declared at the end of the manuscript.

*Correspondence: d.sommers@tue.nl

¹ Mathematics and Computer Science, Eindhoven University of Technology, Eindhoven, the Netherlands

Abstract

The assessment of process mining techniques using real-life data is often compromised by the lack of ground truth knowledge, the presence of non-essential outliers in system behavior and recording errors in event logs. Using synthetically generated data could leverage ground truth for better evaluation. Existing log generation tools inject noise directly into the logs, which does not capture many typical behavioral deviations. Furthermore, the link between the model and the log, which is needed for later assessment, becomes lost. We propose a ground-truth approach for generating process data from existing or synthetic initial process models, whether automatically generated or hand-made. This approach incorporates patterns of behavioral deviations and recording errors to produce a synthetic yet realistic deviating model and imperfect event log. These, together with the initial model, are required to assess process mining techniques based on ground truth knowledge. We demonstrate this approach to create datasets of synthetic process data for three processes, one of which we used in a conformance checking use case, focusing on the assessment of (relaxed) systemic alignments to expose and explain deviations in modeled and recorded behavior. Our results show that this approach, unlike traditional methods, provides detailed insights into the strengths and weaknesses of process mining techniques, both quantitatively and qualitatively.

Keywords: Evaluation, Validation, Synthetic process data, Realistic noise, Behavioral patterns

Introduction

A process mining assessment method should include *validation* to check the result for correctness, quantitative and qualitative *evaluation* to assess the effectiveness and accuracy, *reliability* assessment to look into repeatability and reproducibility of the results, *robustness* assessment to assess the ability of the method to cope with variations in the data and noise, *performance* and *scalability* assessment to assess efficiency and ability to cope with large and complex logs and models, and *usability* assessment to evaluate interpretability of obtained results (Aalst et al. 2012).

Already in 2008, Rozinat et al. identified the need for a common framework for evaluating process mining results (Rozinat et al. 2008). They emphasized that such a framework should help researchers compare algorithm performance and end users to validate

their results. They also stressed that such a framework should allow users to influence process and log characteristics and support the generation of “forbidden” scenarios as a complement to the actual execution log. Despite the progress in the process mining field and the availability of tools for model generation, simulation, and injection of noise into event logs, the challenges formulated in Rozinat et al. (2008) still remain relevant.

We encountered these challenges when working on the NWO CERTIF-AI project, in which we developed alignment methods for the conformance checking of processes involving multiple entities, such as objects and resources performing different tasks, taking the interaction of these entities into account in the alignments. We obtained data from real-life processes with varying characteristics and noise levels. However, when evaluating our alignment methods to check whether the recorded behavior of a manufacturing process from our partner company Omron is compliant with a prescriptive model provided by process owners, we found it difficult to validate the results without major efforts from stakeholders and time-consuming manual inspection, making a large scale evaluation and validation practically impossible. The exposed individual deviations in process behavior can be explained in several ways by e.g., timing issues in the recording, multitasking by human operators, or operators temporarily switching roles.

To validate the results, we would need to have knowledge about the true causes of deviations, be it recording errors or behavioral outliers. To conduct a proper large-scale validation we would need multiple models with varying characteristics, multiple executions of those models with different violations of the prescribed behavior, and multiple event logs with recording errors. In addition to that we need an “omniscient” stakeholder, an “oracle” able to tell whether the deviation detected in the alignments is a true deviation and whether we provide a correct explanation of this deviation.

It is clearly infeasible to provide such a “ground truth oracle” when using real-life data, especially if the goal is to do experimentation on a large scale. Therefore, we resort to using synthetically generated process models and event logs, with realistic characteristics, both in terms of recording errors and behavioral deviations. Several tools like PTAndLogGenerator (Jouck and Depaire 2019), PUPRLE (Burattin et al. 2022), and AIR-BAGEL (Ko et al. 2020) make steps towards this goal. However, these inject noise directly into simulated logs, which does not allow for generating data containing typical behavioral deviations observed in real-life processes and validating whether the results of the process mining methods can handle such deviations correctly.

In this paper, we propose an approach for generating synthetic data in which realistic noise is incorporated into a simulation model with the help of model transformations capturing patterns of both behavioral deviations and recording errors, resulting in an imperfect process model and an event log obtained for this model. Both the transitions of the model, representing “true” events and deviations, and the event records in the log contain tags indicating the deviation type, thus allowing to implement an oracle aware of behavioral outliers and recording errors. Using this approach, we describe the creation of datasets for synthetic processes with different characteristics and illustrate the use with an alignment validation example.

This paper is organized as follows. In “[Related work](#)” section, we discuss related work. In “[Requirements to synthetic process data](#)” section, we discuss the impact of the characteristics of real-life data on the assessment and pose our research question concerning

how this can be mimicked synthetically, providing knowledge of the ground truth. We describe our method and discuss how its produced components contribute to our goal in “[Generating process data with ground truth](#)” section, followed by a description of the current tool support in “[Tool support](#)” section. “[Putting the framework into practice](#)” section demonstrates the approach describing the complete process of creating datasets of synthetic process data and “[Evaluating the interpretability of \(Relaxed\) multi-object alignments](#)” section demonstrates the approach showing validation results and generated insights. We discuss the implications of our work in “[Conclusion and future work](#)” section.

This paper is an extension of work originally presented in ICPM (Sommers et al. [2024](#)) in which we generalize the proposed approach by generating a set of event logs, instead of a single one, that can be generated from every subset of selected patterns and simulation parameters to provide a range of represented characteristics. We provide three instances of using the proposed approach resulting in three datasets of synthetic process data, each of which is characterized by recording errors and behavioral deviations that cannot be established using existing synthetic data generation methods. This illustration complemented by a description of the current status of tool support is also aimed at increasing the accessibility of our approach. Further, we elaborate on the components of the approach regarding the patterns and model transformations through examples, and we discuss the impact of the simulation parameters affecting the simulated event log.

Related work

In this section, we give a brief overview of assessment practices in process mining with real-life event logs and synthetic data.

Assessment using real-life logs with unknown model

Evaluating process mining techniques using real-life data requires significant effort and active participation of data owners. The case study with a UWV event log in Dees et al. ([2017](#)) is an excellent example of this type of work. Although such case studies provide invaluable insights and help identify strengths and limitations of process mining techniques, the proprietary nature of this data often prevents it from being shared with the process mining community for future research. The assessment of repeatability and reproducibility of the studies becomes out of reach.

An established practice for evaluating process mining techniques is to use open data sets provided in yearly Business Process Intelligence Challenges (BPIC) from 2011 to 2020 (Dongen [2011](#), [2012](#), [2014](#), [2015](#); Dees and Dongen [2016](#); Dongen [2017](#); Dongen and Borchert [2018](#); Dongen [2019](#), [2020](#)) and the Conformance Checking Challenges (CCC) from 2019 and 2020 (Munoz-Gama et al. [2019](#); Buijs [2014](#)), which originate from real-life processes and can be found in the [4TU Data Repository](#). Some of these data sets have become very popular, with BPIC 2012 being cited in nearly 500 papers and BPIC 2017 by almost 300, according to Google Scholar at the time of writing this paper. From the very first years, BPIC event data was realistically noisy in terms of both its recording and behavior (Bose et al. [2013](#)), which made this data important for testing and comparing various techniques in real-life scenarios.

A complicating factor in assessing process discovery techniques with BPIC data is that the ground truth knowledge about the underlying process is missing, with only limited information available thanks to efforts in understanding the processes through (manual) analyses and consultations with domain experts. Assessing the quality of such process mining techniques as conformance checking or model repair using BPIC or CCC data remains a complex and not straightforward task, as the corresponding behavioral models are not known. Even when a prescriptive process model is available, as in Munoz-Gama et al. (2019), it is infeasible to determine whether deviations between the modeled behavior and the recorded behavior are correctly exposed and explained, as they could be caused by different behavioral deviations as well as recording errors, and the correctness of their identification by a conformance checking algorithm cannot be verified.

To address the question of whether process mining methods work on real-life small, incomplete event logs, Käppel et al. (2021) developed an evaluation framework that reduces event logs and generates small event logs by removing traces either randomly or along the time dimension, producing training and test logs. Removal of traces can lead to side effects related to the workload of resources, case interactions, etc., making the log less realistic. In a sense, this approach moves towards evaluation with synthetic data.

Assessment with synthetic data

Besides real-life data, synthetic data is used to provide datasets for evaluating process mining methods. A number of tools were developed for this purpose. *PTAndLogGenerator* (PTALG) described in Jouck and Depaire (2019) is a tool that allows to generate a population of non-structured process models with user-defined probabilities for sequence, choice, parallel, and loop structures. Corresponding event logs consisting of single object traces are obtained by simulation. ‘Noise’ is imputed after simulation directly into traces and includes missing head, missing body (episode), missing tail, order perturbation, and the introduction of additional activities, adopting the definition of noise from Günther (2009), additionally incorporating a randomly generated decision model and data attributes.

In Burattin et al. (2022), a PURPose-Guided Log gEneration (PUPRLE) framework is proposed to produce event logs with different properties for targeting different mining purposes. It takes a process model, not restricted to its modeling language, to simulate an event log through (guided) execution of the model. Depending on the purpose of the synthetic data, the simulated event log can be attributed with different characteristics. E.g., for its use in process discovery and conformance checking, the resulting recorded behavior includes some predefined infrequent behavior, and noise, as defined in Günther (2009), respectively. Note that this noise is again limited to the same simple log manipulations as in PTALG, and is only imputed after simulation. Furthermore, the input process model represents the actual behavior of the process, i.e., it includes behavioral outliers contributing to infrequent behavior.

AIR-BAGEL (Ko et al. 2020) is an interactive tool designed to inject pseudo-real anomalies into event logs by associating them with specific root causes, such as resource behavior or system malfunctions, using a probabilistic mechanism for resource and systems errors, resulting in e.g. skipping a step or rework. The tool generates logs

augmented with labels and attributes indicating the corresponding anomaly types, enabling the evaluation of event log cleaning methods.

Figure 1a provides a simplified illustration for these approaches where a process model M_0 is assumed to describe the true behavior of the process, and used to simulate a “clean” event log L . Noise is imputed only afterwards by a log manipulation function ϵ^L . Figure 1b shows our approach, which we introduce in “Generating process data with ground truth” section.

The availability of such tools facilitates large-scale evaluations of process mining techniques. For example, Van Houdt et al. (2024) generated 400 artificial event logs using PTALG to generate models, BPSimPy to populate models with simulation parameters like the total simulation length, and L-Sim simulator to generate low-level event logs. The empirical evaluation of unsupervised log abstraction techniques presented in Van Houdt et al. (2024) primarily focuses on the precision and fitness of discovered models and provides interesting insights related to the influence of abstraction techniques used on the balance between fitness and precision. They note that increasing the scale of experimentation allowed them to obtain more nuanced results.

Requirements to synthetic process data

To conduct an assessment of process mining techniques on synthetic but realistic data, we should be able to generate synthetic process data including (1) process models with different process characteristics, like the degree of parallelism or non-determinism, as they may unknowingly influence the results, (2) variation in deviations/noise, both in process execution and its recording, consistent to real-life deviation patterns, and (3) a ground truth which provides a target function for the assessment problem at hand.

Process models representing processes with different characteristics can be generated using existing model generation tools, like PTALG. In this paper, we address requirements (2) and (3) only, assuming that we already have models representing the system, automatically or manually generated.

As a running example, we manually create a synthetic package delivery process. An abstract, simplified overview of the process is illustrated in Fig. 2, starting with the choice of *home* or *depot* delivery, after which the package (black) queues for a *warehouse employee* (green) to *pick* and *load* it into a *van* (yellow). In case of home delivery, a *courier* (blue) drives off and *rings* a door after which he continues to either immediately hand over the package (deliver home), or deliver it at the corresponding *depot* (red) after



(a) Synthetic process data generation methods (b) Generation of synthetic process data with behavioral and recording deviation patterns Π^S and Π^L and model transformation function Ψ

Fig. 1 Approaches to generating synthetic data (model M and log L) for an underlying process S

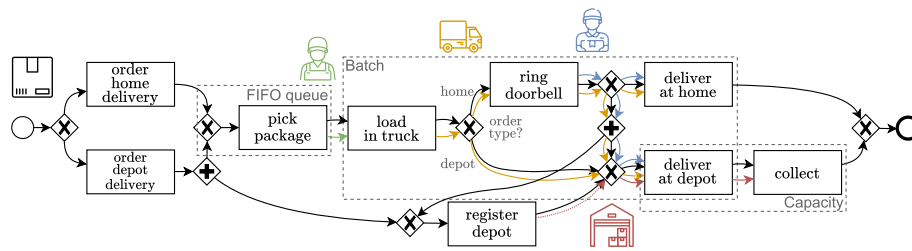


Fig. 2 Abstract process model for the package delivery process, including modeled behavior of *packages* (black), *FIFO queue* (yellow), *warehouse employees* (green), *delivery vans* (orange), *deliverers* (blue), and *depots* (red)

Table 1 Categorization of data quality issues for each entity in recorded behavior, taken from Bose et al. (2013)

	Case	Event	Belongs to	C_attribute	Position	Activity name	Timestamp	Resource	E_attribute
Missing data	RI_{mi}^C	RI_{mi}^E	RI_{mi}^O	RI_{mi}^{Ca}	RI_{mi}^P	RI_{mi}^a	RI_{mi}^t	RI_{mi}^O	RI_{mi}^{ea}
Incorrect data	RI_{in}^C	RI_{in}^E	RI_{in}^O	RI_{in}^{Ca}	RI_{in}^P	RI_{in}^a	RI_{in}^t	RI_{in}^O	RI_{in}^{ea}
Imprecise data			RI_{im}^O	RI_{im}^{Ca}	RI_{im}^P	RI_{im}^a	RI_{im}^t	RI_{im}^O	RI_{im}^{ea}
Irrelevant data	RI_{ir}^C	RI_{ir}^E							

registration, where it is left for *collection*. Alternatively, for depot delivery, ‘ringing’ and therefore also ‘deliver home’ is omitted in the subprocess.

Characteristics of real-life processes and their data

We need mechanisms to generate data with typical *recording errors* and *behavioral outliers*, with the type of deviation clearly indicated for later use in the assessment.

We consider recording errors in event logs as records misrepresenting the actually executed behavior of the process. In Bose et al. (2013), *recording errors* are subdivided into four categories of issues, where data could be (1) missing, (2) incorrect, (3) imprecise, and (4) irrelevant. Each category could apply to various elements of the event log, e.g., events, event attributes, relations between attributes and events, and between events themselves. Table 1 summarizes these quality issues. Possible sources of recording errors are faulty logging mechanisms, either automatic or manual, with unsynchronized clocks, too coarse timestamps, data corruption, and filtering and aggregation methods.

Recent advancements (Basmer et al. 2024) extend these quality issues to object-centric event data, further classifying the issues on object-centric properties like object types and their relations. While this fits the processes we consider in this work, the essence of the data quality issues is the same. All issues from Table 1 are represented in the set of processes from the BPIC datasets (Bose et al. 2013), and could be reflected in the running example of the synthetic package delivery process. For the examples that follow we focus primarily on the categories of *missing* and *incorrect* data.

A process model ties together a set of modeling patterns (Russell et al. 2006) to define the “main” behavior of the process. The interpretation of the “main behavior” depends

on the process mining task. It could be the expected behavior, the “happy flow” behavior, or frequent behavior. *Behavioral outliers* refer to behavior that deviates from the *main behavior* of the process. There could be several reasons for violating the modeled behavior, e.g., people not following guidelines, or encountering rarely seen cases not taken into account in the model, thus choosing different behavioral patterns than the ones imposed by the model.

The abstract process model from Fig. 2 includes various modeling patterns, like choices, parallelism, queueing, batching, capacity, resource memory, and (continuous) correlation, involving several objects simultaneously. Behavioral deviations are considered patterns as well, which in essence boil down to similar issues as the recording errors when they are captured in the event log. Later, we explicitly draw connections between the two types of issues to signify this relation.

Process mining techniques should be robust to recording errors and should be able to handle behavioral deviation appropriately (defined e.g., based on their frequencies), depending on the use case. Take for example process discovery. When the discovered process model is to be used as a handbook for employees, the goal might be to capture only the frequent behavior, whereas when the process model is to be used as a digital twin, the goal is to capture all possible behavior.

When resorting to the use of synthetic data, these deviation characteristics should be mimicked as closely as possible to represent real-life data. Model-log pairs that can be obtained by the approach illustrated in Fig. 1a are not sufficient, as (1) the model of the “real system” is not provided: the base model does not show behavioral deviations, or they are included in the base model and not distinguishable from regular behavior, (2) the log manipulations are performed irrespectively of the behavior of the process, therefore losing the link between model and log.

The need for ground truth

Dealing with these categories of errors and deviations is a challenge for process mining methods in real-life settings in general. This holds even more so for their assessment, as the explanations for causes of deviations may be ambiguous or missing altogether. For the evaluation of process discovery techniques, conformance metrics allow for measuring the quality of a process model where the ground truth model is unknown. However, this assumes the event log is free of recording errors, since differentiating them from behavioral outliers may be impossible due to ambiguity in their causes. While recording errors should ideally be filtered out completely, (some) behavioral outliers should end up in the behavior of the discovered process model. Therefore, for validation, being able to make this distinction is required. Similarly, for conformance checking, assessing whether deviations are correctly exposed and explained, having knowledge of the causes is essential for constructing the target.

Without going into detail, we argue that other PM problems, e.g., decision mining, model or log repair, bottleneck detection, and predictive and prescriptive process monitoring, suffer from the same complications. Therefore researchers are forced to resort to assessing PM techniques using synthetically generated data.

This brings us to our research question (RQ):

RQ: How can we set up an experiment with realistically noisy process data where the ground truth is known to allow for a complete assessment?

Generating process data with ground truth

Depending on the goal of a PM algorithm/tool, and its input and output, its assessment may require a process model, an event log, and the ground truth containing information about both behavioral and recording deviations that are to be used in assessing the quality of the produced results.

In this section, we propose an evaluation framework for generating such process data using a collection of deviation patterns and corresponding model transformations to build in the deviation into an initial process model. We first give a high-level overview of the framework, after which we discuss deviation patterns and model transformations.

Framework design

Our framework is schematically depicted in Fig. 1b. M_0 denotes a base model representing the expected behavior of the process, M^S serves as the true model of the process, capturing behavioral deviations, and M^L extends this model by imitating faulty logging mechanisms in the model. A simulation of M_0 results in an “exemplary” log showing the expected behavior of a process. A simulation of M^S gives a “clean” event log, showing the “real” behavior of a process. Simulating M^L results in log L' capturing the real behavior with realistic recording errors. This log L' is the target of our framework.

M_0 , M^S , M^L , L' together with the links between them provide the ground truth about the process and its execution, including behavioral characteristics of the process and recording characteristics of the event log as well as its completeness with respect to the process (M^S) and the model (M_0).

The base model M_0 might describe an existing process, be a hypothetical process designed manually with certain characteristics in mind, or it can be generated automatically by a model generation tool. Recall that M_0 merely serves as a base process that does not represent the “real” process execution. It can serve as e.g., a normative model in the assessment of conformance checking techniques, or a “typical flow” model in the assessment of process discovery techniques.

We start with a base process model M_0 and a set Π of deviation patterns. Deviations, either recording or behavioral, are modeled as templates. A model transformation function Ψ defines how the behavior modeled in deviation pattern $\pi \in \Pi$ can be added to a model. The deviating behavior is incorporated into the base process model, resulting first in model M_S of process executions with behavioral deviations, and subsequently in model M_L incorporating log recording errors on top of that. By simulating the resulting model M_L , we acquire a realistic event log L' and the knowledge of all deviations that took place, since the simulated events can be linked back to transition firings from M^L , which in their turn can be linked to the transitions of M_S and M_0 , or are labeled as corresponding deviations.

The set Π of deviations consists of subsets Π^S with behavioral deviation patterns and Π^L with recording error patterns. A model transformation function Ψ takes a model M , a deviation pattern $\pi \in \Pi$, and a function h mapping elements of π to the elements of M in order to indicate where the deviation should occur in the model. The model

transformation series starts with M_0 , and after applying the first transformation, applies the next transformation to the obtained $\Psi(M_0, \pi_1, h_1)$, etc., finally leading to the model M^S of the actual process execution. Similarly, we start with M^S to apply patterns from Π^L and obtain M^L after a series of model transformations. When simulated, M^L produces the event log L' , encoding the complete information about the transitions that generated events.

Only partial information on the events (e.g., transition labels) is given to a process mining technique to be assessed. The rest of the information constitutes the ground truth knowledge to be used to compute quality metrics in the assessment and obtain qualitative insights.

Usage in assessments

To use the evaluation framework, assessors must define a ground truth function gt for their PM problem and assessment tasks. This function transforms the combination of M_0 , M^S , M^L , and L' to the target result for that process mining technique. Further on, the assessors are to choose a distance function d to compare the result produced by the process mining technique to the ground truth target. Generically,

$$d\left(f(M_0, L'), gt^f(M_0, L', M^S, M^L)\right)$$

measures the quality of a PM method f , using distance function d and ground truth function gt^f .

Evaluation and validation of *process discovery* (PD) and *process repair* techniques can be performed by comparing discovered models to either M_0 , M^S , and/or M^L , depending on the application at hand. For example, the goal can be to discover an interpolation of M_0 and M^S , including frequent behavioral deviations but ignoring the infrequent ones and all the recording errors. The target process model is defined by M_0 and a subset $\Pi^{S'} \subseteq \Pi^S$ of behavioral deviations. The similarity between a discovered model $PD(L')$ and $gt^{PD}(M_0, M^S, M^L, L')$ can be computed e.g., using model similarity (Dijkman et al. 2011).

The goal of *conformance checking* (CC) with $f = CC$ is to identify and explain deviations in the recorded behavior from the behavior captured by a normative model. A CC technique takes M_0 and L' as input e.g. to compute an alignment of the actual behavior of the process and the modeled behavior. The ground truth target can be defined as the optimal alignment, interpreting events corresponding to the firing of original transitions of M_0 as synchronous moves and events corresponding to the transitions being part of the deviation patterns as log moves or model moves accordingly.

The similarity between the alignment $CC(M_0, L')$ produced by a conformance checking technique and the ground truth alignment $gt^{CC}(M_0, M^S, M^L, L')$ can be computed e.g., by sequence or graph edit distance, depending on the formalization used to model alignments (Yujian and Bo 2007; Gao et al. 2010). Note that this is an abstract definition of gt^{CC} . We concretize this in “[Evaluating the interpretability of \(Relaxed\) multi-object alignments](#)” section, where we use our proposed evaluation framework to evaluate the explainability of multi-object alignments (Sommers et al. 2022) and relaxed multi-object alignments (Sommers et al. 2024).

Deviation patterns

In Table 2, we list a selection of deviation patterns that are currently implemented in our framework. The left-hand columns form the set of issues from Table 1 specified for recording errors, originating from the issues presented in Bose et al. (2013). The right-hand columns present a concrete set of behavioral issues, originating from violations of the modeling patterns discussed in Russell et al. (2006). As discussed in “Characteristics of real-life processes and their data” section, these issues can be similarly categorized as the recording issues. The counterparts are denoted by the colored links between the columns. E.g., for behavioral deviations, there can be several deviations that fall under ‘incorrect object’ (RI_{in}^o), namely switching correlation, multitasking, etc.

Take for example the recording error pattern RI_{mi}^e , describing a missing event, where an activity is executed but failed to be recorded. Skipping an activity during the execution of a process (BI_1) is a behavioral deviation being a counterpart of a missing event: In both cases, the corresponding event will be missing in $\log L'$, although the causes of that would differ.

A deviation pattern consists of a behavioral description and an abstract process model fragment. The abstract process model fragment serves as a blueprint for incorporating the deviating behavior into a process model. Formally, a *deviation pattern* is an abstract process model $\bar{M}$ describing deviating behavior in relation to some process behavior. Abstract means here that certain model elements are represented by wildcards, denoted as $\langle * \rangle$, acting as placeholders for specific elements of the model to which the deviation is to be added.

To illustrate and clarify the use of patterns, we elaborate on the modeled deviation patterns for a few examples from Table 2, using typed Petri nets with Identifiers (t-PNIDs) (van der Werf et al. 2022) as the modeling formalism. This formalism allows for modeling intricate behavior of several interacting objects, corresponding to patterns described in “Requirements to synthetic process data” section. A t-PNID consists of transitions (rectangles), typed places (colored circles), and labeled arcs. A transition can

Table 2 Deviation patterns

Recording error patterns		Behavioral deviation patterns	
Missing event	RI_{mi}^e	BI_1	Switching correlation
Incorrect event	RI_{in}^e	BI_2	Multitasking
Missing object	RI_{mi}^o	BI_3	Skipping activity
Incorrect object	RI_{in}^o	BI_4	Neglecting object(s)
Missing position	RI_{mi}^p	BI_5	Overtaking in queue
Incorrect position	RI_{in}^p	BI_6	In/decreasing capacity
Incorrect activity name	RI_{in}^a	BI_7	Switching roles
		BI_8	Same resource on 4-eyes principle
		BI_9	Different resource on resource memory
		BI_{10}	Ignoring batching
		BI_{11}	Long duration

consume and produce tokens by ‘being fired’, respectively from its incoming places, i.e., with an incoming arc from a place, and to its outgoing places, which are denoted as the transition’s *pre-set* and *post-set*. Tokens in a t-PNID carry (multiple) identifiers representing the object names in the system. The firing of a transition represents an event with the involved objects corresponding to the objects represented by the consumed and produced tokens and potentially enables new transition firings which determines the positions in their ordering. For the formal semantics of t-PNIDs, we refer to van der Werf et al. (2022).

Figure 3 shows examples of abstract modeling blueprints for the following patterns:

RI_{mi}^o The recording error of a *missing object(s)* can be modeled by the abstract t-PNID shown on the top-left of Fig. 3. Transition $\langle t \rangle$ and its pre- and post-set with corresponding arcs (blue) are matched from the base model. Assuming that the place types coincide with the types of the object(s) $\langle O \rangle$ to be missing, a second transition $\langle t \rangle_{\text{missing } \langle O \rangle}$ is created (green) with the same activity name as $\langle t \rangle$ but without the involvement of $\langle O \rangle$. To achieve this behavior, two additional silent transitions $\tau_{\bullet \langle t \rangle - \langle O \rangle}$ and $\tau_{\langle t \rangle \bullet - \langle O \rangle}$ are created, which bypass the tokens from places corresponding to objects from $\langle O \rangle$, i.e., from $\bullet \langle t \rangle$ to $\langle t \rangle \bullet_{\langle O \rangle}$, and subsequently to $\langle t \rangle \bullet$. Tokens from the other places, i.e., $\bullet \langle t \rangle \upharpoonright_{\langle O \rangle^C}$ and $\langle t \rangle \bullet \upharpoonright_{\langle O \rangle^C}$, are passed on to $\langle t \rangle_{\text{missing } \langle O \rangle}$ where $\langle O \rangle^C$ denotes the complement set of $\langle O \rangle$.

RI_{in}^p The recording error of *incorrect position* occurring due to batch logging can be modeled by the abstract t-PNID shown on the middle-left of Fig. 3. This pattern

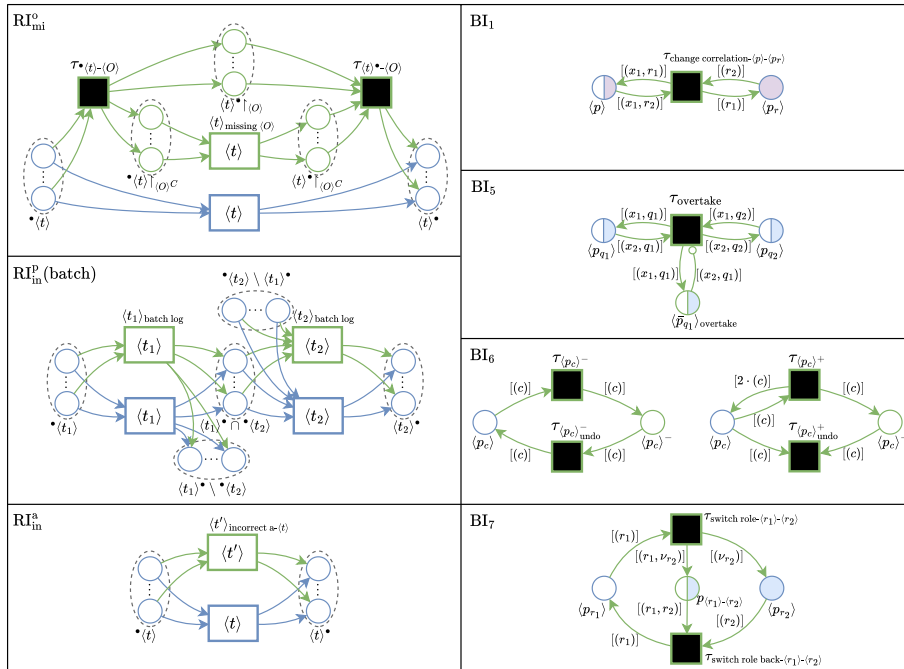


Fig. 3 t-PNID modeling blueprints for three recording issue patterns (left) and four behavioral deviation patterns (right). Blue and green elements respectively correspond to matched and newly created elements

is designed to be applied to a specific batching pattern in the base model (blue), where a first transition ($\langle t_1 \rangle$) initializes work on a batch of objects, and a second transition ($\langle t_2 \rangle$) processes the objects one-by-one. The created elements, i.e., $\langle t_1 \rangle_{\text{batch log}}$ and $\langle t_2 \rangle_{\text{batch log}}$ with the same activity names as $\langle t_1 \rangle$ and $\langle t_2 \rangle$ represent the same behavior. However, by applying different timing specifications, the simulation shows the behavior where $\langle t_1 \rangle_{\text{batch log}}$ takes as long as processing the whole batch and $\langle t_2 \rangle_{\text{batch log}}$ takes a negligible amount of time, representing the same activities but logged at an incorrect position. The distinction between the intersection and differences between the post-set of $\langle t_1 \rangle$ and the pre-set of $\langle t_2 \rangle$ is made to correctly pass along the tokens.

- RI_{in}^a The recording error of an *incorrect activity name* corresponding to an event can be modeled by the abstract t-PNID shown on the bottom-left of Fig. 3. This is a simple pattern, creating a duplicate transition $\langle t' \rangle_{\text{incorrect a-}(t)}$ for a matched transition $\langle t \rangle$ inheriting its pre- and post-set and with a different activity name, specified by $\langle t' \rangle$.
- BI_1 The behavioral deviation of *changing correlation* can be modeled by the abstract t-PNID shown on the top-right of Fig. 3. It consists of two matched places, $\langle p \rangle$ and $\langle p_r \rangle$ (blue) and one created silent transition $\tau_{\text{change correlation-}(p)-(p_r)}$ (green), with wildcards for $\langle p \rangle$ and $\langle p_r \rangle$. $\langle p \rangle$ is assumed to be a place in the base model with two types, of which one corresponds to a resource with the same type as $\langle p_r \rangle$. The silent transition, through the labels on the connected arcs, consumes a correlation token from $\langle p \rangle$ and a resource token from $\langle p_r \rangle$ to change the correlation to the new resource. In Sommers et al. (2024), a similar pattern, falling under the same behavioral deviation, is presented which swaps the resources from two existing correlation tokens instead of changing the correlation for one token.
- BI_5 The behavioral deviation of *overtaking* in a first-in-first-out queue can be modeled by the abstract t-PNID shown on the upper-middle-right of Fig. 3. A created silent transition τ_{overtake} swaps the correlation from two objects in matched places $\langle p_{q_1} \rangle$ and $\langle p_{q_w} \rangle$ with their respective queue objects denoting the place in the queue, similarly as the previous pattern. Additionally, a created place $\langle p_{q_1} \rangle_{\text{overtake}}$ is added to prevent a continuous cycle of objects overtaking each other in the simulation of the model.
- BI_6 The behavioral deviations of *decreasing* and *increasing* the capacity of a resource object can be modeled by the abstract t-PNID shown on the lower-middle-right of Fig. 3. A created silent transitions $\tau_{\langle p_c \rangle -}$ decreases the capacity of an object in matched place $\langle p_c \rangle$, stores this information in a created place $\langle p_c \rangle^-$ which can subsequently be undone by silent transition $\tau_{\langle p_c \rangle^- \text{undo}}$. Similarly, to increase the capacity, $\tau_{\langle p_c \rangle +}$ duplicates a token in $\langle p_c \rangle$, which can be undone by $\tau_{\langle p_c \rangle^+ \text{undo}}$ which consumes the duplicated tokens.
- BI_7 The behavioral deviation of *switching roles* of a resource object can be modeled by the abstract t-PNID shown on the bottom-right of Fig. 3. Two matched places $\langle p_{r_1} \rangle$ and $\langle p_{r_2} \rangle$ correspond to places where resource objects with two different roles reside. A created silent transition $\tau_{\text{switch role-}(r_1)-(r_2)}$ can move a token representing a resource object from role $\langle r_1 \rangle$ to the role of $\langle r_2 \rangle$. Similarly to the previous pattern, this information is stored in place $p_{\langle r_1 \rangle - \langle r_2 \rangle}$ to undo this move

with created silent transition $\tau_{\text{switch role back}\langle r_1 \rangle - \langle r_2 \rangle}$. Note that the ν variables in the arc labels ensure that the objects from different types are non-overlapping, by synthesizing a new identifier that references the corresponding object from role $\langle r_1 \rangle$.

We refer to Sommers et al. (2024) for four more examples of the patterns RI_{mi}^e , RI_{in}^e , BI_3 , BI_2 , and BI_1 in which two correlations are swapped, i.e., slightly different from BI_1 as presented here.

Our approach is defined conceptually, independent of the modeling formalism, as long as the formalism of the abstract deviation patterns, both for behavioral outliers and recording errors, matches that of the base model. Therefore, it is not restricted to the use of t-PNIDs.

The list can be extended with additional pattern descriptions and corresponding definitions. All provided patterns are modeled in a way that the model transformation is additive in behavior, i.e., behavior described in the pattern is added to the base model without eliminating any behavior that was captured in the model. For this reason, multiple deviation patterns do not interfere with each other. The created elements have distinct names. This is a necessary property to ensure that the resulting process model allows sufficient freedom to define ground truth functions for different process mining tasks.

Model transformations

Behavior from a deviation pattern π , described by an abstract process model M_π , can be incorporated into a process model M , through a model transformation function Ψ which uses M , M_π , and an injective mapping function h mapping the wildcards from π to elements in M .

Our definition for Ψ is derived from Petri net transformations as discussed in Ehrig and Padberg (2003) and is defined as follows:

$$\Psi(M, \pi, h) = M \cup h(\pi) \quad (1)$$

where $h(\pi)$ is a morphism from the abstract process model fragment π to a concrete process model fragment in the space of M , with *matched elements* referring to elements in M . Through Ψ , $\Psi(M, \pi, h)$ additionally contains the *created components* of π , providing the added behavior.

To illustrate how patterns from the examples in Fig. 3 are applied to a process model, we show in Fig. 4 a fragment of the t-PNID implementation M' of a slightly modified version of the abstract process model from Fig. 2 for the running example. This is the same formalism as the examples presented for some of the recording

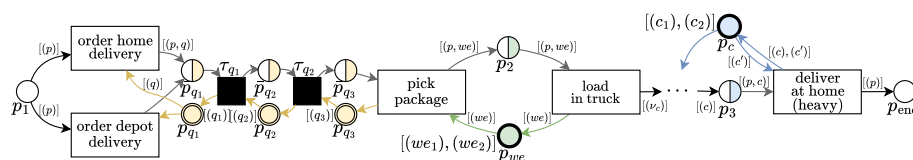


Fig. 4 t-PNID fragment M' of a modified version of M , with the addition of a heavy delivery at home

error and behavioral deviation patterns. Additionally, not shown in the patterns, the process' initial state is denoted by the multisets of object names $[(we_1), (we_2)]$ and $[(c_1), (c_2)]$ in places with a thicker outline, i.e., p_{we} and p_c , modeling warehouse employees and couriers. To apply the pattern for a missing recorded object, the process is extended with an additional activity for *delivery of heavy packages at home*, requiring two couriers for its execution.

Figure 5 depicts the applied model transformations of four patterns, i.e., *overtaking* (BI_5), *changing correlation* (BI_1), *switching roles* BI_7 , and *missing recorded objects* (RI_{mi}^o), to M' . The pattern BI_5 for overtaking from Fig. 3 is applied with mapping $\{\langle p_{q_1} \rangle \mapsto \bar{p}_{q_2}, \langle p_{q_2} \rangle \mapsto \bar{p}_{q_3}\}$, to create a silent transition connecting to these existing places from the base model M' which can be used to swap the correlation between two objects and their place in the queue. Similarly, the pattern BI_2 for changing correlation from Fig. 3 is applied to base model M' with mapping $\{\langle p \rangle \mapsto p_2, \langle p_r \rangle \mapsto p_{we}\}$ where p_{we} is the resource place for warehouse employees. The created silent transition can be used to change the correlation of the employee loading a package in a truck after it is picked by a different employee. The pattern BI_6 for switching roles is applied M' to temporarily switch the role with the created silent transition of a warehouse employee to that of a courier, by the mapping to the corresponding resource places, i.e., $\{\langle p_{r_1} \rangle \mapsto p_{we}, \langle p_{r_2} \rangle \mapsto p_c\}$.

Lastly, the pattern RI_{mi}^o is applied M' to model the recording of a heavy delivery with a missing second courier. Note that through the created silent transition, both couriers are claimed throughout the execution of the created heavy delivery transition, which however is unaware of the second involved courier, as it is bypassed through the created place p'_c .

Note that there are restrictions on the mapping of the wildcards to correctly model the behavioral pattern into the base model. Places and transitions have specific roles in the net, which should match the roles of the to-be-matched elements from a pattern. This could be either checked in the model transformation, or the user should be instructed clearly on the requirements of the wildcards. E.g., the multitasking pattern is defined behaviorally that the correlation is prematurely destroyed, putting a resource from busy to idle. This requires that $\langle p_1 \rangle$ and $\langle p_2 \rangle$ are mapped respectively to a correlation/busy place of a resource role and its corresponding idle place.

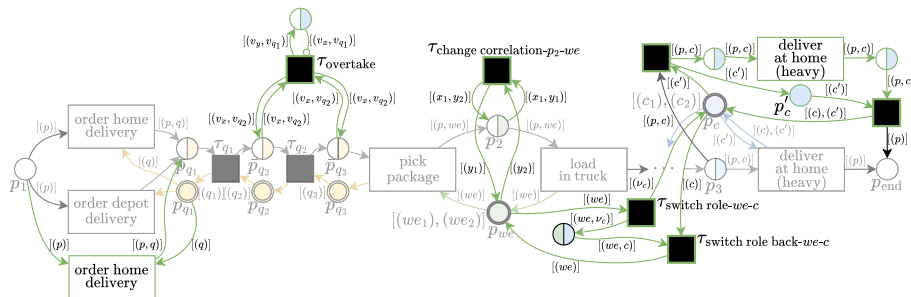


Fig. 5 The result of model transformations applied to M'

Simulation

The event log L' , which is the last element generated by the framework, describes the recorded behavior of the process. L' is obtained by discrete event simulation of the process model M^L , which represents the real behavior of a process including behavioral outliers (by Π^S) and faulty recording mechanisms (by Π^L), deviating from the expected behavior modeled in M_0 .

The simulation module is an interchangeable part of the framework. E.g., L' can be obtained by a play out of M^L , allowing for repeated firings of transitions, either through being enabled, or scheduled in the form of arrivals or predetermined schedules.

For a more advanced simulation method, information about stochastics and timing could be included in the process model. Example modeling formalisms are Stochastic Petri nets (Leemans et al. 2021) and Timed Petri net (Razouk and Phelps 1983; Zuberek 1980). We explain the concepts here without going into the formal definition of these formalisms.

Adding weights to transitions of a net provides probabilities to transition firings. A transition firing can be sampled from a categorical distribution for all enabled transitions with corresponding bindings/modes, with probabilities defined by the transition weights. For example, with enabled transition firings T_μ^e and weight function $w : T \rightarrow \mathbb{R}_0^+$, the probability that the selected transition firing, denoted as t_μ^s , is $t_\mu \in T_\mu^e$ is defined by

$$p(t_\mu^s = t_\mu | w) = \frac{w(t)}{\sum_{t'_\mu \in T_\mu^e} w(t')} \quad (2)$$

A transition firing consumes tokens from incoming places and produces tokens in outgoing places. Until now, this has been defined atomically. Delaying the production of tokens adds time specifications and durations to the simulation of the execution of the process model. The duration of the delay is specified on transitions or, more granularly, on outgoing arcs of transitions. Transitions are withheld from consuming tokens during the delays of the corresponding production in places. There are various ways to define the time specification, e.g., by constants, functions, or distributions, as long as the (sampled) delay is non-negative. Let us say for example that the time it take from picking a package and loading it in a truck is normally distributed with a mean and variance of respectively of fifteen and six minutes. The duration can be specified by $+N(15,6)$ on the arc from transition ‘pick package’ to place p_2 in the t-PNID model depicted in Fig. 4.

Both the transition weights and time specifications allow for additional conditional parameters, by adding a variable denoting e.g., the current marking of the net, the *current simulation time* $\eta \in \mathcal{T}$, or other data attributes (Holliday and Vernon 1987). With $w : T \times \mathcal{T} \rightarrow \mathbb{R}_0^+$, specific transitions can be temporarily disabled, which is especially useful for controlling probabilities and frequencies of deviating behavior during simulation. Similarly, for time specifications, e.g., the variance of the resources’ productivity can be more accurately modeled based on the current simulation time.

In “Deviation patterns” section, we define deviation patterns, both for behavioral outliers and recording errors, on the behavior of the modeling formalism. Note that with extended modeling formalisms for more advanced simulation, like SPNs and TPNs, deviations can be naturally extended to the stochastics of the net. Two examples of this

are the pattern RI_{in}^p for batch logging as described in “[Deviation patterns](#)” section and BI_{11} for long durations from Table 2. In both patterns, the behavior does not change from the perspective of the workflow, however, the timing specifications characterize the deviating behavior during simulation. As such, deviations on the stochastics, timings, and other external data attributes can be modeled directly in the sampling distributions for weights and durations based on conditionals like time or markings.

Independently of the specific simulation module, the simulated behavior of the process model M^L resembles real-life behavior by design, as it incorporates both behavioral deviations and recording errors.

Tool support

We provide two versions of implementations of the evaluation framework, as proposed conceptually above. One version is a tool, called Trident, with a graphical user interface (Sommers et al. 2023), offering a user-friendly interaction but is limited in functionality. This version can be operated through either a Flask GUI, a command line interface, and/or run from other Python code, to generate ground truth synthetic process data. The source code, an installation manual, and a screencast of example usage are available at gitlab.com/vignesh_dv/mira/-/tree/paper/mira/pattern.

The other version contains the source code implementation with example usage for more advanced deviation patterns and simulation with handling of stochastics and time durations (Sommers et al. 2024). Also based on Python, this version can be operated by creating scripts invoking the appropriate classes and definitions. The source code and usage instructions are available at gitlab.com/dominiquesommers/mira/-/tree/main/mira/simulation.

Both implementations operate on regular Petri nets (Peterson 1981; Murata 1989), ν -nets (Rosa-Velardo and de Frutos-Escrig 2010), Resource-Constrained ν -nets (Sommers et al. 2022, 2023) (RC ν -nets), and t-PNIDs, and can be extended to other formalisms. One of which is a generalization of t-PNIDs for handling variably involved objects for modeling one-to-many interactions, similarly as Object-centric Petri nets with Identifiers (OPIDs) (Gianola et al. 2024), Object-centric Petri nets (OC nets) (Aalst and Berti 2020), and Synchronizing proclets (Fahland 2019). The framework can therefore also be used to generate logs of object-centric processes. This allows to generate rich logs of processes with event records including information about resources that participated in the activities’ executions.

The user interface of Trident is focused towards resource-constrained ν -nets (Sommers et al. 2022), distinguishing between the place types being regular, resource available, and resource busy. Recall from “[Model transformations](#)” section that the model transformations put requirements on proper mapping of elements from the abstract deviation pattern to the base model. These requirements can be easily checked, due to the strictly defined modeling restrictions of RC ν -nets.

In general, the usage for both tools is similar and follows the concept as described in “[Framework design](#)” section. i.e., one takes a real-life process model or sketches a hypothetical process and a set of appropriate deviations. The deviation patterns from Table 2, are provided and already defined. Not being restricted to those, additional patterns can be defined to see fit. The selected deviations are applied to the base model through a

series of model transformations, and the resulting process model M^L can be either simulated directly or alternatively exported for usage in an external simulation module.

The simulation module included in Trident is a simple play out of M^L from the provided initial to final marking, with the option to set a limit of the number of transition firings, in case of infinite behavior. Probabilities are modeled through sampling a waiting time for transition firings from the moment they are enabled. In case the transition is not enabled anymore at the scheduled time, its firing is canceled. The simulation is basic in terms of probabilities of which transitions to fire, i.e., it does not take into account other dependencies than the sampled scheduling time from the moment it is enabled.

The more advanced implementation provides a simulation module with sampling of transition firings and duration of token productions as described in “Simulation” section. Furthermore, the arrivals of spontaneous objects and the start/stop schedules of scheduled objects can be defined by sampling distributions. The conditional variable of the current simulation time can be used to e.g., vary the probability of deviating behavior over time.

If one requires a more advanced simulation, an export of process model M can be used in other tools.

Putting the framework into practice

Starting from a single base model M_0 for a (synthetic) process, the framework, as described in “Framework design” section and illustrated in Fig. 1b, can be used to generate a versatile dataset of event logs, with different, controllable characteristics. In general, any combination of mapped behavioral deviation patterns can be applied to M_0 , creating a set of n models for M^S , with n the number of such combinations. Subsequently, the same can be done for mapped recording error patterns, on each M^S , resulting in a set of $n \cdot m$ models for M^L , with m the number of combinations of recording errors. Lastly, with k different simulation parameters, $n \cdot m \cdot k$ logs can be generated from M^0 , each with their own distinct characteristics with regard to included behavioral deviations, recording errors, and stochastic. Figure 6 shows this extended usage of the framework.

In this section, we provide three instances of using the framework as described by Fig. 6 to create synthetic datasets of processes following the description of “Framework

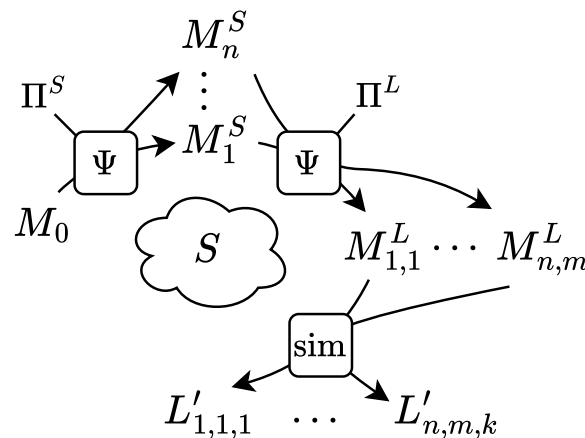


Fig. 6 Framework extension to generate multiple event logs from a single base model

design” section. For each, we describe the complete process, i.e., from designing a base model, to selecting and mapping deviation patterns, to choosing simulation parameters to generate several event logs. We reflect on their properties and compare qualitatively the characteristics of the datasets with regard to the capabilities of other synthetic data generation methods from “Assessment with synthetic data” section. All models and corresponding generated logs with the applied patterns are available at gitlab.com/dominiquesommers/mira/-/tree/main/mira/simulation.

Package delivery process

The first instance follows from the running example of the package delivery process (c.f., Fig. 2) used throughout the previous sections. This synthetically designed process includes several modeling patterns as described in “Characteristics of real-life processes and their data” section. Let us say that various recording mechanisms are incorporated to record the activities, e.g., orders are registered in an information system, and the activities in which human actors are involved are logged by them manually scanning the packages’ barcodes. For the delivery at a depot, the corresponding depot is registered by the courier.

Note that this process is designed in a way such that it could indicate a real-life process. We deem this an important property as to reflect realistic characteristics.

For this demonstration, we formalized this process as a t-PNID which serves as the base process model M_0 , and describes the expected behavior of the package delivery process.

Behavioral deviations: The following behavioral deviation patterns with corresponding mappings are applied to M_0 as illustrated by the purple deviations in Fig. 7:

- BI_5 Overtaking in the FIFO queue for picking packages;
- BI_7 Switching roles from a courier to that of a warehouse employee;
- BI_{10} Batching is ignored, leaving with a delivery van before it was fully loaded;
- BI_3 Skipping the activity of ringing, modeling behavior where e.g., the door was already opened upon arrival;
- BI_9 Different resource memory where the package is delivered to a different depot than where it is registered;
- BI_2 Multitasking of couriers during the delivery of multiple packages, modeling interruption of a delivery.

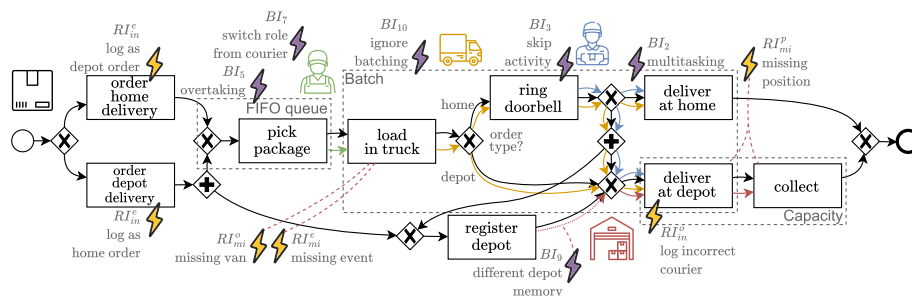


Fig. 7 Abstract and simplified base process model for the package delivery process from Fig. 2, with additionally annotated the added behavioral outliers (purple thunderbolts) and recording errors (yellow thunderbolts)

For this dataset, we choose to isolate the patterns, not combining them within a single simulated event log. We therefore create six versions for M^S , each applying a single behavioral deviation pattern model transformation from the list above. Additionally, we add a version for M^S , denoted as $M_\emptyset^S$ without behavioral deviations.

For recording errors, the following patterns with corresponding mappings are applied to $M_\emptyset^S$, illustrated by the yellow deviations in Fig. 7:

- $RI_{in}^e(1)$ *Incorrect event*, recording an *order for depot delivery* when it was intended for *home delivery*;
- $RI_{in}^e(2)$ *Incorrect event*, vice versa, i.e., recording an *order for home delivery* when it was intended for *depot delivery*;
- RI_{mi}^e *Missing event* for the activity of *loading* a package in a truck;
- RI_{mi}^o *Missing object* of the involved *van* for *loading*, e.g., due to a temporary connection failure of a recording device;
- RI_{in}^o *Incorrect object* of the *courier* when *ringing*, e.g., due to not logging out by the courier on the previous shift;
- RI_{mi}^p *Missing positions* for the recording of the *delivery* and the *collection* at a depot, e.g., due to coarse timestamp logging.

For the other versions of M^S , no recording error patterns are applied. This results in a set of twelve models representing M^L , six of which with a distinct behavioral deviation pattern and six of which with a distinct recording error pattern. For this process, we select a single set of simulation parameters that ensure that the patterns are invoked in the simulation of a small number of packages (two) in each of these twelve models, as illustrated in Fig. 8. The twelve generated event logs each contain the behavior for handling the two packages with the corresponding issue, be it either a behavioral deviation or a recording error.

Note that it is actually not inherently infeasible to generate the same (or similar) event logs as we described here, using the existing synthetic process data generation methods from “[Assessment with synthetic data](#)” section. For each behavioral deviation pattern, a separate process model can be constructed, and similar recording errors can be introduced, either with careful settings regarding the noise injection, or manually. However, we argue that creating the dataset following our framework provides a methodological and structured approach to its design, where the incorporated issues stem from the literature on real-life process data. As the patterns for both behavioral deviations and recording errors are designed to be additive in behavior, the expected behavior of the process does not change. Next, we demonstrate using our framework for creating another dataset with deviation patterns that are not directly supported by the existing methods “[Assessment with synthetic data](#)” section.

Energy contract process

For our second dataset, we design again a synthetic process originating from a process mining course given at our university. The process considers the handling of energy contracts and is illustrated in Fig. 9. It is in terms of structure and involved

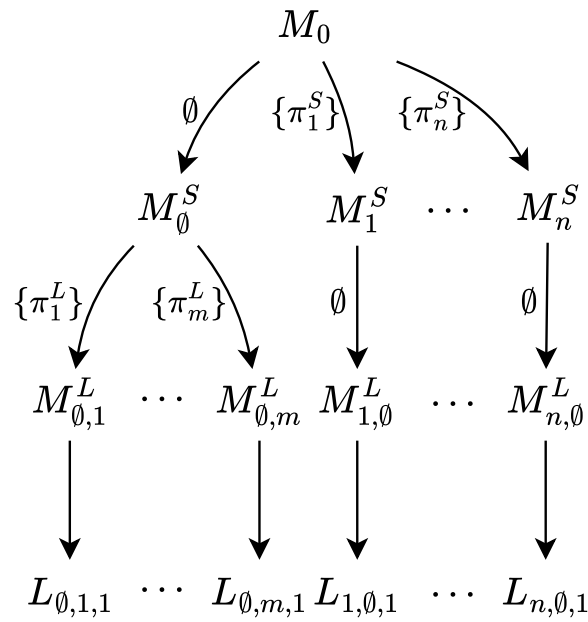


Fig. 8 Framework as applied to the synthetic package delivery process

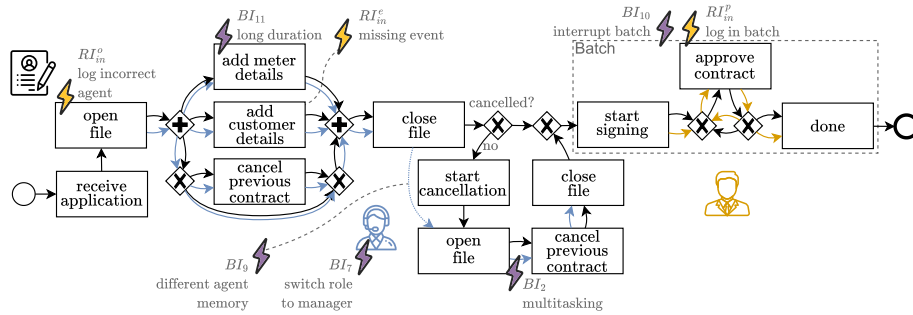


Fig. 9 Abstract and simplified base process model for the energy contract process from, with additionally annotated the behavioral outliers (purple thunderbolts) and recording errors (yellow thunderbolts) that are applied

objects less complicated than the package delivery process, however, it includes some additional properties.

Customers can apply for a new energy contract online, after which a few pre-checks are performed by the company. After the pre-checks when *receiving* the application, the employees in the back office *add the customer details* to the database. They also *add the meter details*. As a service, the previous contract of the customer is *canceled*. Finally, a manager does a final check and *approves* the contract. We see three object types in this process: one for *applications* (black), one for the *call agents* (blue) in the back office, and one for *managers* (orange).

In the subprocess of adding the details, which are executed in parallel by a single agent, the call agent is expected to work uninterruptedly on an application, which starts and ends with opening and closing the file. Cancellation of the previous

contracts is either done during this subprocess or in a later stage, for which the agent reopens the file, resulting in activities with duplicate labels in the base process model.

Managers are expected to approve contracts in batches. Batching is defined slightly differently here than in the package delivery process, where the delivery can only be initiated after a batch is complete, i.e., a van is fully loaded. Here, a manager can initiate the handling of contracts at any point in time (*start signing*) and they finish the approvals of each contract in the taken batch before resuming with newly arrived contracts.

The model from Fig. 9 is again formalized as a t-PNID and serves as the base process model M_0 , describing the expected behavior of the energy contract process.

We add the following behavioral deviation patterns with corresponding mappings to M_0 as depicted by the purple deviations in Fig. 9:

- BI_7 *Switching roles* where an *agent* temporarily takes over the role of the *manager*;
- BI_9 *Different resource memory* where a different *agent* does the second phase *cancellation*;
- BI_{10} *Ignore batching of approving contracts*, interrupting the handling of a batch and contracts are added to it before the batch finishes;
- BI_2 *Multitasking of agents* in the second phase *cancellation*, e.g., in case where the first phase cancellation requires more attention than the second;
- BI_{11} *Long duration of adding meter details* taking exceptionally long with regard to the usual processing times.

We combine each deviation pattern in one model, creating a single version for M^S , to which we add the following recording error patterns with corresponding mappings, to create a single version for M^L , as depicted by the yellow deviations in Fig. 9:

- RI_{mi}^e *Missing event* for the activity of *adding customer details*;
- RI_{in}^o *Incorrect object* of the involved *agent* in the first phase of handling applications, e.g., due to not logging out by the agent on the previous shift;
- RI_{in}^o *Incorrect object* where the activity of *cancel previous contract* in the first phase is accompanied with the data label of *false*, possibly resulting in a second cancellation;
- RI_{in}^p *Incorrect position* due to batch logging of *approving contracts*, causing a batch of contracts to be approved seemingly at the same time.

M^L contains each deviation pattern described above, which, with simulation parameters, is ready for simulating corresponding event logs. We select simulation parameters such that the frequencies of the occurrences of issues are low, and set the a duration distribution for deviation BI_{11} such that it deviation from the existing activity for adding meter details. By simulating the process for a large number of contract applications, we assume that by the law of large numbers, each issue will appear in the generated logs with approximately the expected frequencies. Note that guided simulation approach like PURPLE (Burattin et al. 2022) (c.f., “[Assessment with synthetic data](#)” section) can complement the simulation model in our framework to ensure characteristics of the event log regarding the frequencies of included issues.

Some of the incorporated issues are applied to activities with duplicate labels in the base model. Differentiating between such activities, and only applying certain issues on one of them, is infeasible without the connection of the simulated recorded events and the process model(s). Therefore, noise injection by random imputations performed on a simulated event log without deviations does not support these issues. Also due to the duplicate labels, the base model is not discoverable by process discovery methods like the Alpha Miner (van der Aalst et al. 2004) or the Inductive Miner (Leemans et al. 2013), even for simulated event logs without any deviations. Hence, we need the corresponding models, i.e., M_0 , M^S , and M^L , for the assessment with a dataset like the one created for this energy contract process.

Real-life assembly process

The third dataset revolves around the real-life manufacturing process from our partner company Omron that we referred to in “Introduction” section. This demonstrates the usage of the framework in a different way, for which the base process model already exists in the form of a normative model and we anticipate issues in the process, both for behavioral deviations and recording errors. Recall from “Introduction” section that discrepancies between the model and event data of this process, e.g., exposed through alignments, can be explained in various ways with no way of verifying the explanation. However, if we add patterns for the different explanations to the base model, we can generate the corresponding scenarios synthetically.

Figure 10 shows an abstract and simplified model of the manufacturing process. This model is created in collaboration with the process owners and acts as the base model M_0 in the framework. *Products* (black) are assembled in seven sequential stages, i.e., stage A to stage G, each containing a predefined sequence of activities. Three operators are responsible for their own stages, i.e., stages A and G for operator 1 (blue), stages B and F for operator 2 (red), and stages C, D, and E for operator 3 (orange). This is based on the physical layout of the stages in the factory, which is roughly resembled by the layout of M_0 in Fig. 10. A stage can only process one product at the same time, as depicted by the capacities. In between stages, there is always the option for a product to be put back to any of the previous stages, through (a series of) reversions.

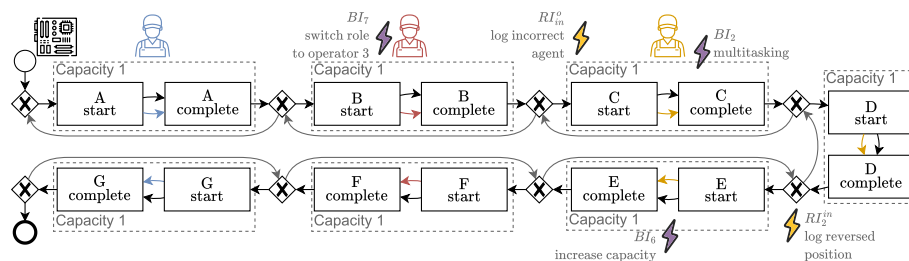


Fig. 10 Abstract simplified normative process model for a real-life assembly process

Activity executions in this process are recorded by manually scanning the corresponding product at the start and completion of an activity. The involved operator object is added automatically and is decided by the hardcoded responsibilities for activities.

For completeness, we consider all seven stages, however for conciseness, we add patterns to half of the process representing the explanations for deviations from “[Introduction](#)” section. Note that it can be trivially extended to the other stages and operators as well. We add the following three behavioral deviation patterns to M_0 to create M^S and subsequently two recording error patterns to M^S to create M^L with corresponding mappings:

- BI_6 *Increase capacity of stage E*, allowing for more than one product to be handled simultaneously;
- BI_7 *Switching roles from operator 2 to operator 3*, e.g., to help out when fully occupied;
- BI_2 *Multitasking of operator 2*, handling multiple products in various stages simultaneously;
- RI_{in}^o *Incorrect object of the involved operator* which is inherently accompanied by the combination of switching roles, due to the working of the recording mechanism. Recall that to the involved operator is added automatically, hence activities in stages C to E involving operator 2 are automatically logged on operator 3.
- RI_{in}^p *Incorrect position for the recording of activities in stages D and E*, caused by manual logging of these activities which is sensitive to incoherent positioning of the recorded events as opposed to when they were actually executed.

For the dataset described here, we simulate M^L for this process using standard simulation parameters to include the ‘what-if’ scenarios caused by the included issues. Note that one can add contextual information and data to the conditions of stochastics encoded in the simulation model to increase the authenticity of the occurrences of the patterns, e.g., using the current state to increase the probability of switching roles when operator 3 is busy. Furthermore, stochastic information from the previously recorded data can be used to ultimately create an event log that is realistic with regard to the underlying real-life process.

The patterns of the issues applied to the base process model are all behaviorally influencing each other, which is not something that is supported by the existing methods from “[Assessment with synthetic data](#)” section.

Evaluating the interpretability of (Relaxed) multi-object alignments

As mentioned in “[Related work](#)” section, alignments are used in conformance checking to expose deviations and provide possible explanations for discrepancies between a process model and a recorded event log. Here, we demonstrate our ground truth assessment approach for qualitative validation and evaluation of three alignment techniques on the synthetically designed package delivery process and the corresponding generated event logs as described in “[Package delivery process](#)” section. This provides a concrete example of the usage of the framework for an assessment of process mining techniques, following the description from “[Usage in assessments](#)” section.

Experimental setup

The assessment question (AQ) we aim to answer is threefold:

AQ1: *Are behavioral outliers and recording errors detected correctly?*

AQ2: *How well do the generated alignments reflect the ground truth explanations?*

AQ3: *How do the behavioral outliers and recording errors affect the computation times of the alignment methods?*

With six behavioral deviation patterns and six recording error patterns, the dataset created from the synthetic package delivery process contains twelve versions of M^L and twelve corresponding event logs, each containing a single issue. This isolates the deviation patterns with each event log handling two package orders using the same delivery van object. The ground truth oracle provides explanations by knowledge of which transitions have fired in the simulation to produce the events in the logs.

M_0 and the event logs serve as input for the three alignment methods we consider in this assessment. These are (1) traditional *per-object alignments* (γ^o), operating on each object in isolation, both in event log and model, ignoring any interaction between objects (Carmona et al. 2018), (2) *systemic alignments* (γ), aligning the event log in its entirety to the process model taking into account violations (Sommers et al. 2022), and (3) *relaxed systemic alignments* ($\tilde{\gamma}$), extending their regular counterpart by allowing for relaxations of objects' interactions both in the log and model (Sommers et al. 2024).

Each method takes an event log being a partially ordered set and a normative model as input. The log used for the alignments is a projection of the simulated logs with some types of deviating events (like skipped steps or missing events) removed from it. The produced alignment contains synchronous moves (on which the log and model agree), log moves (indicating that an activity from the log cannot be mimicked in the model) and model moves (indicating which events required by the model are missing in the log).

Results and discussion

The detected deviations for each log are summarized in Table 3. Each row in the table corresponds to a log. The Dev. column indicates the deviation used in the corresponding log. The place of the deviations in the model is shown in Fig. 7. The cells of the table specify the set of objects included in log and model moves of the corresponding alignments, and in addition the set of affected objects in brackets (in the subscript) where applicable. The right-most column summarizes the interpretations of the results.

For recording errors, each deviation was detected by each method, however, there is a difference in the involved objects when the deviation occurs on object level rather than event level. For the single-object alignment γ^o , we see only the deviation's responsible object, for the system alignment γ , we see all objects involved in the corresponding activity, and for the relaxed systemic alignment $\tilde{\gamma}$, we see a combination of the two, correctly separating responsible objects from affected objects. The last recording error (RI_{mi}^p) is not considered a deviation in the context of alignments. From the computed alignments for $RI_{in}^e(1)$ (c.f., Fig. 11), we see that none of the methods resolve the deviation correctly, as all conclude that the *ring* event is incorrectly logged instead of the *order depot* event. The sub-alignments clearly show how each method distinguishes between the involved objects.

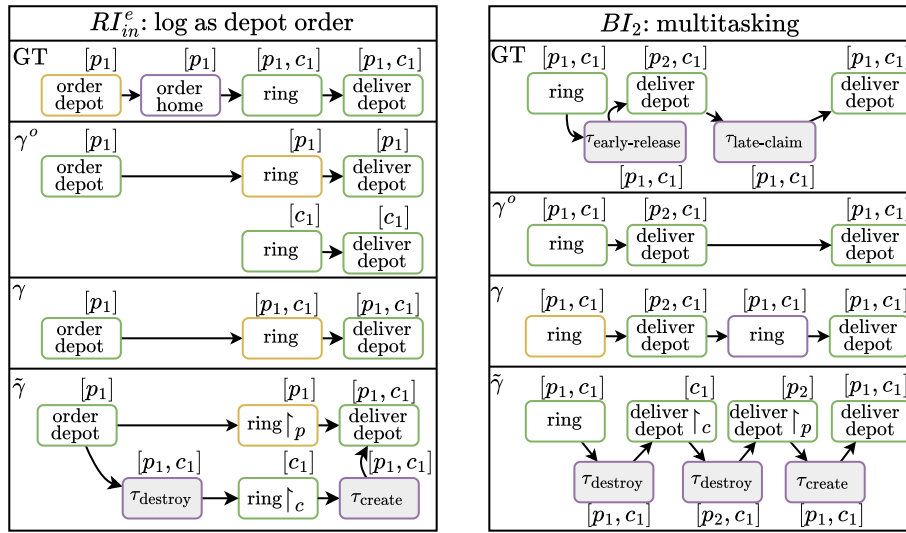


Fig. 11 Ground truth (GT) and computed (γ^o , γ , and $\tilde{\gamma}$) sub-alignment results for recording error pattern RI_{in}^e and behavioral deviation pattern BI_2 . Log, model, and synchronous moves are colored yellow, purple, and green respectively. For clarity, truck and depot objects are omitted

For behavioral outliers, the two deviations BI_7 and BI_3 occur on the level of isolated activity or object and are detected similarly as the recording errors discussed above. The others are not detected by single-object alignments, as they correspond to deviations in objects' interactions, which are not considered. Except for BI_9 , where the executed behavior of both depot objects is incomplete according to the model.

Systemic alignments γ expose all deviations, again noting all involved objects without indication of the object(s) responsible for the deviation. This distinction is made by relaxed alignments $\tilde{\gamma}$ for deviations BI_7 and BI_3 , while for the remaining deviations, the involved objects are detected without log and (labeled) model moves.

From the alignments computed for BI_2 (c.f., Fig. 11), we can see the consequence of ignoring correlations of objects, as the single-object alignments γ^o show no deviations. This is resolved in the systemic alignment γ . However, its explanation differs from the deviation pattern used to generate the log, contrary to the relaxed systemic alignment $\tilde{\gamma}$, which includes (relaxed) synchronous moves for each recorded event and alterings of correlations. Note that these alignments still allow for variance in their interpretation, e.g., the separated synchronous moves for deliver depot and the non-matching silent model moves.

Figure 12 shows the durations of computing the alignment for the three methods on the log for each pattern of recording errors and behavioral deviation. Looking at the graph in the top row, we can compare the computation times for the three methods for each pattern. The results distinguish three 'patterns' regarding the increase in computation time from the less to the more demanding techniques: (1) For each pattern except for BI_5 , BI_{10} , and BI_7 , we see an exponential increase for the three methods. (2) For BI_5 and BI_{10} , the computation time increases similarly from object alignments to systemic alignments, but there is no significant increase to relaxed alignments. (3) For BI_7 , there is no significant increase in computation time between any of the three methods.

Table 3 Experiment results showing the objects responsible for the deviation and those affected by it (in subscript) for each pattern, for the ground truth (GT) and detected by alignment methods: per-object (γ^o), systemic (γ), and relaxed systematic ($\tilde{\gamma}$)

Dev.	Deviating object(s)				Interpretation of the results
	GT	γ^o	γ	$\tilde{\gamma}$	
$RI_m^e(1)$	$\{p\}$	$\{p\}$	$\{p, c\}$	$\{p\}_{(\{c\})}$	Responsible object is exposed by each, γ and $\tilde{\gamma}$ incorrectly add the courier as responsible and affected object
$RI_m^e(2)$	$\{p\}$	$\{p\}$	$\{p, q\}$	$\{p\}$	Responsible object is exposed by each, γ incorrectly cascades the deviation to the queue
RI_m^e	$\{p, v, e\}$	$\{p, v, e\}$	$\{p, v, e\}$	$\{p, v, e\}$	Responsible objects are exposed by each
RI_m^o	$\{v\}_{(\{p, e\})}$	$\{v\}$	$\{p, v, e\}$	$\{v\}_{(\{p, e\})}$	Responsible object is exposed by each, $\tilde{\gamma}$ correctly pinpoints the affected objects, not exposed by γ^o and considered as responsible by γ
RI_m^o	$\{c\}_{(\{p\})}$	$\{c\}$	$\{p, c\}$	$\{c\}_{(\{p\})}$	Same as RI_m^o
RI_m^p	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	Not a deviation in context of alignments
BI_5	$\{q\}_{(\{p\})}$	$\emptyset$	$\{p, q, d\}$	$\emptyset_{(\{p, q\})}$	Affected objects exposed by γ and $\tilde{\gamma}$, without adding correct context. Not exposed by γ^o
BI_7	$\{c\}_{(\{p, v\})}$	$\{c\}$	$\{p, c, v\}$	$\{c\}_{(\{p, v\})}$	Same as RI_m^o
BI_{10}	$\{v\}_{(\{p, c\})}$	$\emptyset$	$\{p, c, v, d\}$	$\emptyset_{(\{c, p, v, d\})}$	Same as BI_5
BI_3	$\{p, c\}$	$\{p, c\}$	$\{p, c\}$	$\{p, c\}$	Same as RI_m^o
BI_9	$\{d\}_{(\{p, c\})}$	$\{d\}$	$\{d, p\}$	$\{d\}_{(\{p, c\})}$	Similar as RI_m^o , and γ missing affected courier object
BI_2	$\emptyset_{(\{c, p, v, d\})}$	$\emptyset$	$\{p, c, v, d\}$	$\emptyset_{(\{c, p, v, d\})}$	Affected objects correctly exposed by $\tilde{\gamma}$, which are considered as responsible by γ . Not exposed by γ^o

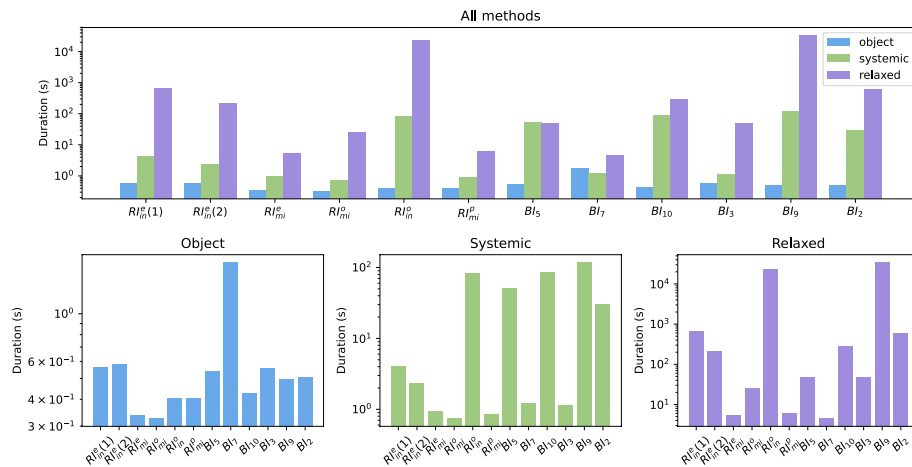


Fig. 12 Computation times of each method for every pattern of recording errors and behavioral deviations

The graphs on the bottom row isolate the data for the three different alignment techniques, showing that the computation times for object alignments are similar across the patterns, with one exception for switching roles (BI_7). Systemic alignments show two patterns, where the computation is either fast or significantly slower. The graph shows more variance for relaxed alignments, with a reasonably constant increase in computation time from systemic alignments, apart from the patterns BI_5 , BI_7 , BI_{10} , which is in line with the patterns from the graph in the top row.

In general, to answer AQ1 and AQ2, we see that single-object alignments can pinpoint deviations on the level of the responsible object, but only if they occur within the workflow of the particular object. Systemic alignments are robust in detecting the deviations but fail to provide details for the responsible objects. Relaxed systemic alignments do both, however, not always provide the correct explanation. We see a clear trade-off between interpretable deviations and computation times. To answer AQ3, there are certain patterns, especially for behavioral deviations, where more interpretability can be achieved without significantly giving in on the computation times, e.g., for BI_7 , BI_5 , and BI_{10} .

This helps to outline the directions for future work in this area. Having the ground truth knowledge provided by our framework was indispensable for conducting this assessment.

Conclusion and future work

Where real-life process data provides for valuable assessments of process mining techniques used in realistic scenarios, the absence of an omniscient stakeholder and specific process characteristics may limit the validity and generalizability of the assessment results. The use of synthetic data allows extending the scale of experimentation and creating an oracle generating the ground truth to be used in assessment.

The approach proposed in this paper makes use of deviations in terms of behavioral outliers and recording errors known from the literature and captures them as modeling patterns and corresponding model transformations. Starting from a base model, we generate separate models for an executed process and for recorded process behavior, as well as a

synthetically generated yet realistically imperfect event log. This approach is limited by the assumption that the deviations can be modeled using the same formalism as the initial model, and by the dependency on the simulation method and parameters regarding the frequencies of the incorporated deviations, potentially affecting the realism of the dataset. The list of deviations described (and implemented) in this work is inherently incomplete and is designed to be extended depending on the domain. E.g., it naturally extends to the stochastic and temporal aspects of the process, which can either be modeled in patterns or the simulation module directly.

We described the usage of the framework for creating datasets consisting of multiple event logs from manually designed base models for three synthetic processes. We showed that the characteristics represented by the datasets regarding recording errors and behavioral outliers are not supported by existing synthetic process data generation methods.

Through a demonstration on one of the created datasets, we show that this approach provides validation and qualitative insights into the strengths, weaknesses, and computation times of different alignment techniques for the detection and explanation of deviations in modeled and recorded behavior. The results can be differentiated per deviation type, i.e., recording errors and behavioral outliers, as well as per deviation itself, and tailored towards domain-specific parts of the process. Interestingly, while an exponential increase in computation time is expected across the different alignment techniques, our experiments showed that this is not the case for some of the patterns that are introduced, even though the interpretability did improve.

Whereas in this work, we focused on the *qualitative* aspect of the *evaluation*, this approach trivially extends to the *quantitative* aspect, by providing a broader dataset with varying frequencies of deviating behavior and analyzing the corresponding statistics of the results. Subsequently, it allows for a *reliability*, *robustness*, *performance*, and *usability* assessment, quantitatively as well as qualitatively, providing results with regard to characteristics of the process, i.e., in terms of domain-specific (expected) deviating behavior.

Authors' contributions

All authors contributed equally to writing and reviewing the manuscript.

Funding

This work is done within the project "Certification of production process quality through Artificial Intelligence (CERTIF-AI)", funded by NWO (project number: [17998](#)).

Data availability

No datasets were generated or analysed during the current study.

Declarations

Ethics approval and consent to participate

Not applicable.

Competing interests

The authors of this manuscript were invited by the program chairs of the International Conference on Process Mining 2024 to submit an extended version of their conference paper to be considered for publication in the *Process Science* journal. The submitted manuscript went through a regular review process. Since Boudewijn van Dongen is co-Editor in Chief for *Process Science*, he was not involved in the decision to invite this paper, nor in the journal's review process, nor did he have any influence on the decisions related to this manuscript.

Received: 11 December 2024 Accepted: 18 February 2025

Published online: 20 March 2025

References

- Aalst WMv, Adriansyah A, Medeiros AKAd, Arcieri F, Baier T, Blickle T, Bose JC, Van Den Brand P, Brandtjen R, Buijs JC, et al (2012) Process mining manifesto. In: Business Process Management Workshops: BPM 2011 International Workshops, Clermont-Ferrand, France, August 29, 2011, Revised Selected Papers, Part I 9, Springer, pp 169–194
- Aalst WMv, Berti A (2020) Discovering object-centric Petri nets. *Fundam Informaticae* 175(1–4):1–40
- Basmer M, Kabierski M, Sahling K, Patecka A, Bala S, Mendling J (2024) A classification of data quality issues in object-centric event data. In: ICPM 2024 International Workshops, LNBI, vol 533. Springer Cham
- Bose RJC, Mans RS, van der Aalst WM (2013) Wanna improve process mining results? In: 2013 IEEE Symposium on Computational Intelligence and Data Mining (CIDM), pp 127–134. <https://doi.org/10.1109/CIDM.2013.6597227>
- Buijs J (2014) Environmental permit application process ('wabo'), coselog project. <https://doi.org/10.4121/UUID:26ABA40D-8B2D-435B-B5AF-6D4BFD7A270>
- Burattin A, Re B, Rossi L, Tiezzi F (2022) A purpose-guided log generation framework. International Conference on Business Process Management. Springer, Cham, pp 181–198
- Carmona J, van Dongen B, Solti A, Weidlich M (2018) Conformance checking, vol 56. Springer, Switzerland, p 12
- Dees M, Dongen BFv, (2016) Bpi challenge 2016. UUV. <https://doi.org/10.4121/UUID:360795C8-1DD6-4A5B-A443-185001076EAB>
- Dees M, de Leoni M, Mannhardt F (2017) Enhancing process models to improve business performance: a methodology and case studies. In: Panetto H, Debruyne C, Gaaloul W, Papazoglou M, Paschke A, Ardagna CA, Meersman R (eds) On the Move to Meaningful Internet Systems. OTM 2017 Conferences, Springer International Publishing, Cham, pp 232–251
- Dijkman R, Dumas M, Van Dongen B, Käärik R, Mendling J (2011) Similarity of business process models: Metrics and evaluation. *Inf Syst* 36(2):498–516
- Dongen BFv (2019) Bpi challenge 2019. 4TU.Centre for Research Data. <https://doi.org/10.4121/UUID:D06AFF4B-79F0-45E6-8EC8-E19730C248F1>
- Dongen BFv (2020) Bpi challenge 2020. 4TU.Centre for Research Data. <https://doi.org/10.4121/UUID:52FB97D4-4588-43C9-9D04-3604D4613B51>
- Dongen BFv, (2011) Real-life event logs - hospital log. Eindhoven University of Technology. <https://doi.org/10.4121/UUID:D9769F3D-0AB0-4FB8-803B-0D1120FFCF54>
- Dongen BFv, (2012) Bpi challenge 2012. Eindhoven University of Technology. <https://doi.org/10.4121/UUID:3926DB30-F712-4394-AEBC-75976070E91F>
- Dongen BFv (2014) BPI Challenge 2014. Rabobank Nederland. <https://doi.org/10.4121/UUID:C3E5D162-0CFD-4BB0-BD82-AF5268819C35>
- Dongen BFv, (2015) Bpi challenge 2015. Eindhoven University of Technology. <https://doi.org/10.4121/UUID:31A308EF-C844-48DA-948C-305D167A0EC1>
- Dongen BFv, (2017) Bpi challenge 2017. Eindhoven University of Technology. <https://doi.org/10.4121/UUID:5F3067DF-F10B-45DA-B98B-86AE4C7A310B>
- Dongen BFv, Borchert F, (2018) Bpi challenge 2018. Eindhoven University of Technology. <https://doi.org/10.4121/UUID:3301445F-95E8-4FF0-98A4-901F1F204972>
- Ehrig H, Padberg J (2003) Graph grammars and Petri net transformations. Advanced Course on Petri Nets. Springer, Cham, pp 496–536
- Fahland D (2019) Describing behavior of processes with many-to-many interactions. International Conference on Applications and Theory of Petri Nets and Concurrency. Springer, Cham, pp 3–24
- Gao X, Xiao B, Tao D, Li X (2010) A survey of graph edit distance. *Pattern Anal Appl* 13:113–129
- Gianola A, Montali M, Winkler S (2024) Object-centric conformance alignments with synchronization. International Conference on Advanced Information Systems Engineering. Springer, Cham, pp 3–19
- Günther C (2009) Process mining in flexible environments. Phd thesis, Eindhoven University of Technology, Industrial Engineering and Innovation Sciences. <https://doi.org/10.6100/IR644335>
- Holliday MA, Vernon MK (1987) A generalized timed Petri net model for performance analysis. *IEEE Trans Softw Eng* 12:1297–1310
- Jouck T, Depaire B (2019) Generating artificial data for empirical analysis of control-flow discovery algorithms: A process tree and log generator. *Bus Inf Syst Eng* 61:695–712
- Käppel M, Jablonski S, Schöning S (2021) Evaluating predictive business process monitoring approaches on small event logs. In: Paiva ACR, Cavalli AR, Ventura Martins P, Pérez-Castillo R (eds) Quality of Information and Communications Technology. Springer International Publishing, Cham, pp 167–182
- Ko J, Lee J, Comuzzi M (2020) AIR-BAGEL: An interactive root cause-based anomaly generator for event logs. In: ICPM Doctoral Consortium/Tools, CEUR Workshop Proceedings, vol 2703. Aachen, pp 35–38
- Leemans SJJ, Fahland D, van der Aalst WMP (2013) Discovering block-structured process models from event logs containing infrequent behaviour. In: BPM 2013 Workshops. LNBI, vol 171. Springer, pp 66–78
- Leemans SJ, van der Aalst WM, Brockhoff T, Polyvyanyy A (2021) Stochastic process mining: Earth movers' stochastic conformance. *Inf Syst* 102(101):724
- Munoz-Gama J, Fuente Rd, Sepúlveda M, Fuentes R (2019) Conformance checking challenge 2019 (ccc19). <https://doi.org/10.4121/uuid:c923af09-ce93-44c3-ace0-c5508cf103ad>
- Murata T (1989) Petri nets: Properties, analysis and applications. *Proc IEEE* 77(4):541–580
- Peterson JL (1981) Petri net theory and the modeling of systems. Prentice Hall PTR, Upper Saddle River
- Razouk RR, Phelps CV (1983) Performance analysis using timed Petri nets. In: International Conference on Parallel Processing. IEEE Computer Society Press, pp 126–128
- Rosa-Velardo F, de Frutos-Escrig D (2010) Decision problems for Petri nets with names. arXiv preprint [arXiv:1011.3964](https://arxiv.org/abs/1011.3964)
- Rozinat A, de Medeiros AKA, Günther CW, Weijters AJMM, van der Aalst WMP (2008) The need for a process mining evaluation framework in research and practice. In: ter Hofstede A, Benatallah B, Paik HY (eds) Business Process Management Workshops. Springer, Berlin Heidelberg, Berlin, Heidelberg, pp 84–89

- Russell N, Ter Hofstede AH, Van Der Aalst WM, Mulyar N (2006) Workflow control-flow patterns: A revised view. *BPM reports*, vol 0622. [BPMcenter.org](https://www.bpmcenter.org)
- Sommers D, Sidorova N, Dongen BFv (2022) Aligning event logs to Resource-Constrained Petri nets. *International Conference on Applications and Theory of Petri Nets and Concurrency*, vol 13288. Springer, Cham, pp 325–345
- Sommers D, Sidorova N, Dongen BFv (2023) Exact and approximated log alignments for processes with inter-case dependencies. In: *International Conference on Applications and Theory of Petri Nets and Concurrency*. LNCS, vol 13929. Springer, Cham, pp 99–119
- Sommers D, Sidorova N, Dongen BFv (2024) Conformance checking with model projections - rethinking log-model alignments for processes with interacting objects. In: *International Conference on Applications and Theory of Petri Nets and Concurrency*, vol 14628. Springer, Cham
- Sommers D, Sidorova N, van Dongen BF (2024) Assessing process mining techniques: a ground truth approach. In: *2024 6th International Conference on Process Mining (ICPM)*, pp 41–48. <https://doi.org/10.1109/ICPM63005.2024.10680619>
- Sommers D, Varadarajan DV, Sidorova N (2023) Trident: generating noisy synthetic processes with ground truth. In: *ICPM Doctoral Consortium/Demo*. Springer
- van der Aalst WMP, Weijters T, Maruster L (2004) Workflow mining: discovering process models from event logs. *IEEE Trans Knowl Data Eng* 16(9):1128–1142
- van der Werf JME, Rivkin A, Polyvyanyy A, Montali M (2022) Data and process resonance: identifier soundness for models of information systems. *International Conference on Applications and Theory of Petri Nets and Concurrency*. Springer, Cham, pp 369–392
- Van Houdt G, de Leoni M, Martin N, Depaire B (2024) An empirical evaluation of unsupervised event log abstraction techniques in process mining. *Inf Syst* 121(102):320
- Yujian L, Bo L (2007) A normalized Levenshtein distance metric. *IEEE Trans Pattern Anal Mach Intel* 29(6):1091–1095
- Zuberek WM (1980) Timed Petri nets and preliminary performance evaluation. In: *Proceedings of the 7th annual Symposium on Computer Architecture*. Springer, pp 88–96

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.